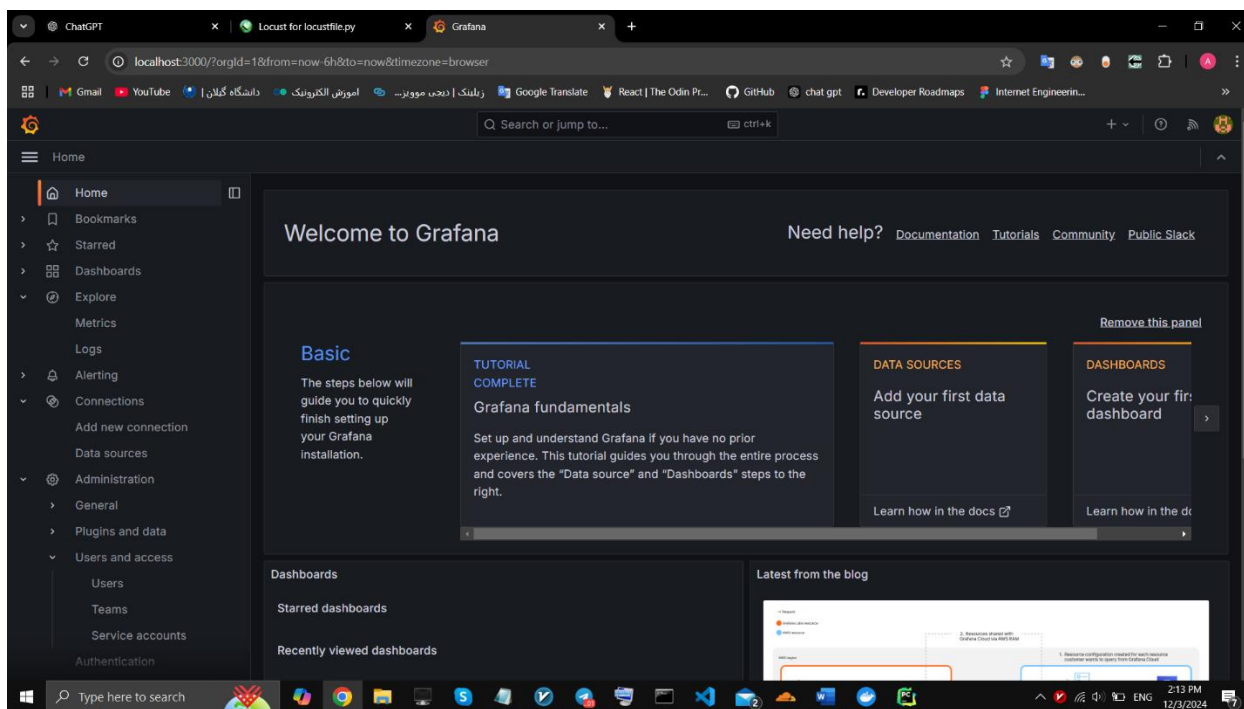


تست کارایی گرافانا با استفاده از لوکاست

گرافانا (Grafana) یک ابزار منبع‌باز برای تجسم، تحلیل، و نظارت بر داده‌ها است که به کاربران اجازه می‌دهد داشبوردهای سفارشی و تعاملی ایجاد کنند. این ابزار از منابع داده متنوعی مانند Prometheus، Elasticsearch، MySQL، InfluxDB و دیگر پایگاه‌های داده پشتیبانی می‌کند و امکان تنظیم هشدارها، کنترل دسترسی، و اشتراک‌گذاری امن داشبوردها را فراهم می‌سازد. گرافانا با قابلیت توسعه‌پذیری و پشتیبانی از پلاگین‌ها، در نظارت بر سیستم‌ها و سرورها، تحلیل داده‌های IoT، مانیتورینگ عملکرد نرم‌افزارها و تجزیه و تحلیل کسب‌وکار کاربرد دارد و به دلیل انعطاف‌پذیری و قدرت بالا، ابزاری محبوب در میان تیم‌های DevOps و تحلیل‌گران داده است.

هدف اصلی **گرافانا (Grafana)** ارائه یک ابزار قدرتمند برای تجسم داده‌ها، نظارت بر عملکرد سیستم‌ها، و تحلیل داده‌های زمانی است. این ابزار به کاربران کمک می‌کند تا داده‌های خام را از منابع مختلف جمع‌آوری کرده و آن‌ها را به صورت گرافیکی، خوانا و تعاملی در قالب داشبوردهای سفارشی نمایش دهند. گرافانا با ایجاد امکان نظارت بر سلامت و کارایی زیرساخت‌ها و اپلیکیشن‌ها، شناسایی مشکلات و ارائه هشدارهای به‌موقع، به بهبود تصمیم‌گیری داده‌محور و افزایش بهره‌وری در سازمان‌ها کمک می‌کند.



تصویری از محیط گرافانا

لوکاست (Locust) یک ابزار منبع باز برای تست بار (**Load Testing**) و تست عملکرد (**Performance Testing**) است که به توسعه دهندگان و تیم های مهندسی اجازه می دهد رفتار سیستم ها و سرویس های خود را تحت فشار بارهای سنگین شبیه سازی کنند. این ابزار به زبان پایتون نوشته شده و امکان تعریف تست های سفارشی از طریق اسکریپت نویسی فراهم است.

کد locustfile.py

```
import json

from locust import HttpUser, task, events

import logging

LOG_FORMAT = ('%(levelname) -10s %(asctime)s %(name) -30s %(funcName) '
              '-35s %(lineno) -5d: %(message)s')

logging.basicConfig(level=logging.INFO, format=LOG_FORMAT)

LOGGER = logging.getLogger(__name__)

class UserBehavior(HttpUser):
    wait_time = lambda self: 1

    @task(1)
    def login(self):
        # Send GET request to fetch the login page and extract any necessary tokens
        response = self.client.get("/login")

        login_data = {
            "user": "admin",
```

```
    "password": "admin"
}
```

```
# Add headers for JSON content type
```

```
headers = {
    "Content-Type": "application/json",
    "Accept": "application/json, text/plain, */*",
    "Origin": "http://localhost:3000",
    "Referer": "http://localhost:3000/login",
}
```

```
# Send POST request with login data in JSON format
```

```
response = self.client.post(
    "/login",
    data=json.dumps(login_data), # Send the data as JSON
    headers=headers
)
```

```
# Log the response details for debugging
```

```
LOGGER.info(f"Login response status code: {response.status_code}")
```

```
LOGGER.info(f"Login response text: {response.text}")
```

```
# Check for successful login by examining the response
```

```
if response.status_code == 200 and "3rafana_session" in response.cookies:
```

```
    LOGGER.info("Login successful")
```

```
else:
```

```
    LOGGER.error("Login failed")
```

```
@task(2)
def index(self):
    self.client.get("/")
```

```
@task(4)
def dashboards(self):
    self.client.get("/dashboards")
```

```
@task(4)
def alerting(self):
    self.client.get("/alerting")
```

```
@task(2)
def search_dashboards(self):
    self.client.get("/dashboards/find?query=my_dashboard")
```

```
@task(3)
def explore_dashboard(self):
    self.client.get("/d/dashboard_uid/my-dashboard")
```

```
@task(1)
def create_dashboard(self):
```

```
self.client.get("/dashboards/new")
```

```
@task(2)
```

```
def alert_rules(self):
```

```
    self.client.get("/alerting/rules")
```

```
@task(2)
```

```
def data_sources(self):
```

```
    self.client.get("/datasources")
```

```
@task(1)
```

```
def create_alert(self):
```

```
    self.client.get("/alerting/alerts/new")
```

```
# Event hooks for logging
```

```
@events.request.add_listener
```

```
def request_handler(request_type, name, response_time, response_length, **kwargs):
```

```
    LOGGER.info(f'Request: {request_type}, Name: {name}, Response Time: {response_time},  
Response Length: {response_length}')
```

```
@events.test_start.add_listener
```

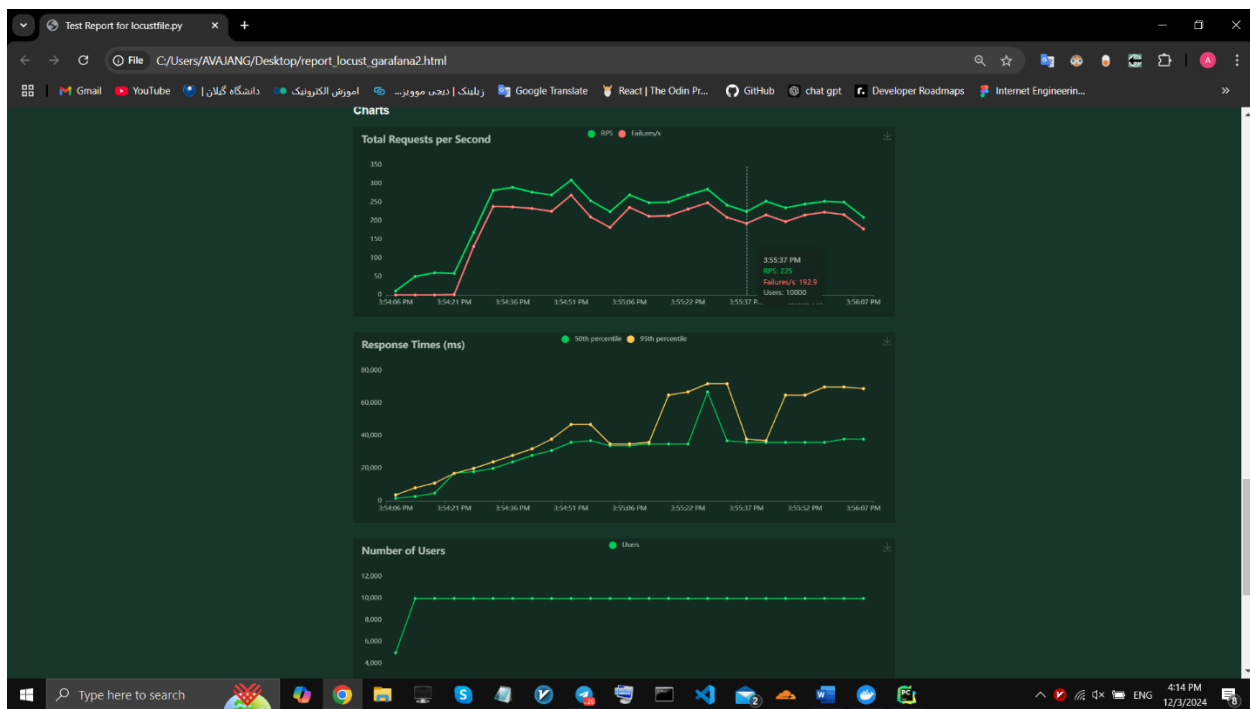
```
def test_start(**kwargs):
```

```
    LOGGER.info('Test started!')
```

```
@events.test_stop.add_listener
```

```
def test_stop(**kwargs):  
    LOGGER.info('Test stopped!')
```

```
@events.quitting.add_listener  
def quitting(**kwargs):  
    LOGGER.info('Quitting fired!')
```



Test Report for locustfile.py

C:/Users/AVA/ANG/Desktop/report_locust_garafana2.html

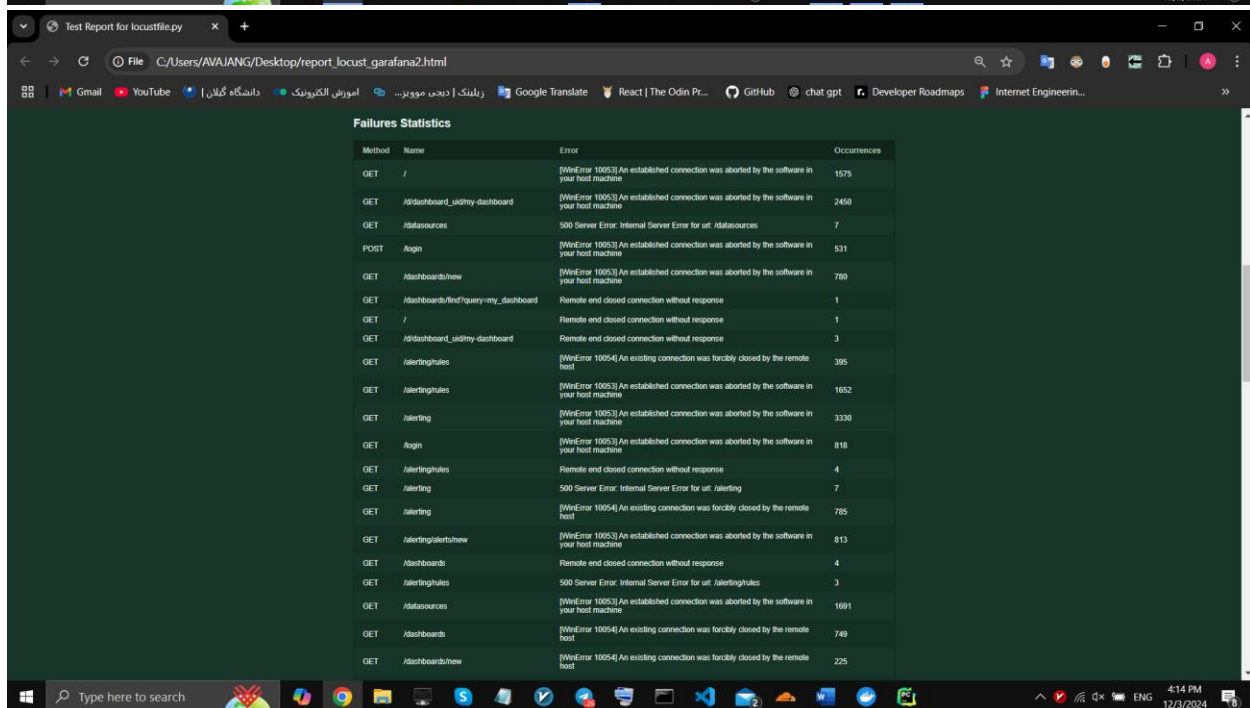
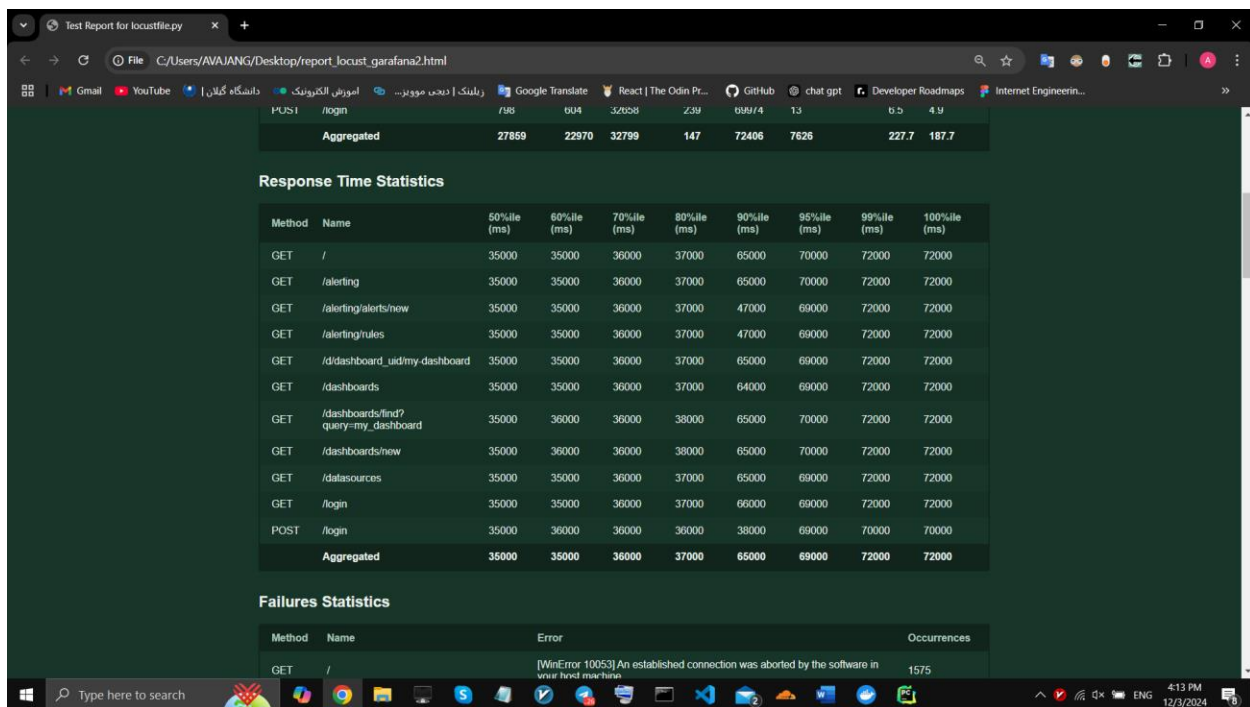
Locust Test Report

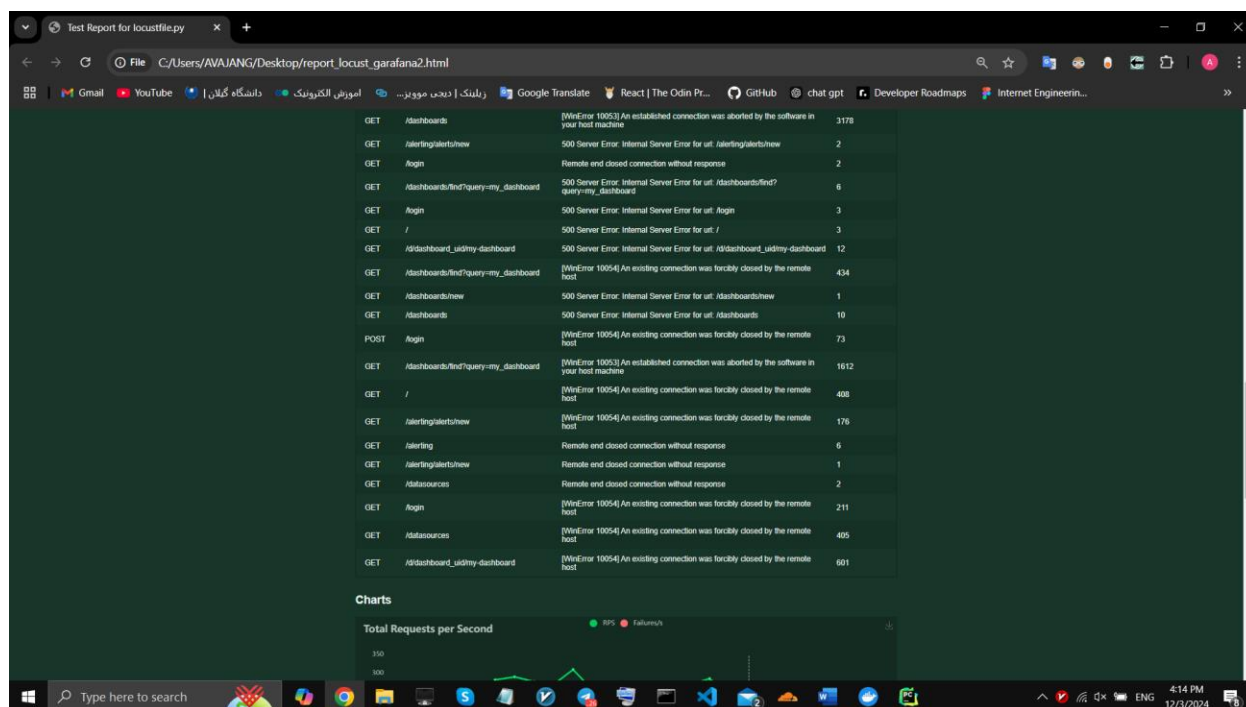
During: 12/3/2024, 3:54:01 PM - 12/3/2024, 3:56:04 PM
Target Host: http://localhost:3000
Script: locustfile.py

Request Statistics

Method	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	/	2435	1987	32893	170	72404	8346	19.9	16.2
GET	/alerting	4966	4128	32801	309	72405	7612	40.6	33.7
GET	/alerting/alerts/how	1208	992	32344	178	72390	8079	9.9	8.1
GET	/alerting/rules	2489	2054	32491	259	72388	7912	20.3	16.8
GET	/d/dashboard_uid/my.dashboard	3712	3066	32728	307	72406	7891	30.3	25.1
GET	/dashboards	4799	3941	32434	168	72403	8098	39.2	32.2
GET	/dashboards/find?query=my_dashboard	2434	2053	33728	319	72405	7105	19.9	16.8
GET	/dashboards/new	1212	1006	33794	283	72388	7692	9.9	8.2
GET	/datasources	2560	2105	32505	300	72401	8034	20.9	17.2
GET	/login	1246	1034	33201	147	72397	7658	10.2	8.4
POST	/login	798	604	32658	239	69974	13	6.5	4.9
Aggregated		27859	22970	32799	147	72406	7626	227.7	187.7

Response Time Statistics





WinError 10053: An established connection was aborted by the software in your host machine

- **علت:** این ارور به این معنی است که ارتباط TCP که از طریق شبکه برقرار شده، توسط نرم افزار در سیستم شما قطع شده است. ممکن است این مشکل ناشی از یک برنامه یا تنظیمات فایروال باشد که به طور ناگهانی ارتباط را قطع می کند.

WinError 10054: An existing connection was forcibly closed by the remote host

- **علت:** این ارور زمانی ظاهر می شود که سرور در اینجا، Grafana یا سرویس مشابه اتصال شبکه را به طور غیرمنتظره ای قطع می کند. این ممکن است به دلایل مختلفی مانند مشکلات در سرور یا بار زیاد بر روی سرور باشد.

500 Server Error: Internal Server Error

- **علت:** این ارور به معنای یک خطای داخلی در سرور است. معمولاً این مشکل ناشی از پیکربندی نادرست یا خطا در کد سرور مثلاً در Grafana است که باعث عدم توانایی سرور در پردازش درخواست ها می شود.

Remote end closed connection without response

- **علت:** این ارور نشان می دهد که سرور به طور غیرمنتظره ای ارتباط را بسته است بدون اینکه پاسخی ارسال کند. این معمولاً به دلیل مشکلات سرور یا بار زیاد بر روی سرور رخ می دهد.

تحلیل و بررسی

گرافانا با تعداد کابر کم مشکل خاصی نداشت و ارور و Failure برای request ها وجود نداشت بنابراین تعداد کاربران را افزایش دادم تا تست فشار بر روی سرور گرفته شود و عملکرد سرور در هنگام تعداد کابر بالا مورد بررسی قرار بگیرد.

با $\text{number of user} = 10000$ و $\text{spawn rate} = 1000$ تست انجام شد.

همان طور که از نمودار request per second مشخص است تا تعداد ۱۰۰ درخواست بر ثانیه هیچ Failure نداریم.

نرخ شکست ها نیز با افزایش نرخ درخواست ها بالا رفته است، به طوری که همزمان با رسیدن به اوج RPS ، شکست ها به بیش از ۲۰۰ درخواست در ثانیه می رسند.

این نشان می دهد که سرور تحت فشار زیادی قرار گرفته و نتوانسته درخواست های جدید را به درستی مدیریت کند.

پس از رسیدن به اوج RPS ، نرخ درخواست ها و شکست ها به حالت نسبتاً پایداری رسیده اند، اما نرخ شکست همچنان بسیار بالاست.

سرور توانایی پردازش تعداد زیادی درخواست در بازه کوتاه را ندارد، که به افزایش شکست ها منجر شده است.

منابع سخت افزاری یا پایگاه داده ممکن است نتوانند به اندازه کافی پاسخگو باشند.